

Требования к конечному проекту

(сетевое многопользовательское взаимодействие)

Цель проекта

Создать визуально и технически завершённый многопользовательский проект, демонстрирующий понимание принципов сетевой архитектуры, синхронизации данных, удалённых вызовов процедур и серверной логики с использованием Unity Netcode for GameObjects (NGO) или FishNet/Photon.

Проект должен быть уникальным — то есть не повторять учебные примеры, представленные на занятиях, а содержать собственные механики, идею или визуальный стиль, делающие его самостоятельной разработкой.

Проект должен быть стабильным (без критических багов и вылетов), поддерживать одновременную игру минимум двух игроков и иметь интуитивно понятный пользовательский интерфейс.

Обязательные требования

1. Сетевая архитектура и подключение

- Выбрана и обоснована сетевая топология: Host-Client (P2P через Relay) или Dedicated Server.
- Корректно настроен NetworkManager.
- Реализовано UI-меню подключения с кнопками Host/Client (или Server/Client для Dedicated Server).
- Игроки успешно подключаются и видят друг друга в общей сцене.

2. Синхронизация данных (State Sync)

- Используются NetworkVariable (NGO) или SyncVar/SyncType (FishNet) для синхронизации состояния игроков или аналог Photon.
- Синхронизированы ключевые параметры: никнейм, здоровье (HP), очки или иные игровые характеристики.
- Изменения состояния корректно отображаются на UI всех подключённых клиентов.
- Трафик оптимизирован: данные не отправляются каждый кадр без необходимости.

3. Движение и владение объектами (Ownership)

- Реализовано сетевое движение через NetworkTransform (NGO) или аналог FishNet/Photon.
 - Локальный игрок управляет только своим персонажем; удалённые игроки отображаются корректно.
 - Камера привязана к владельцу и не переключается на чужих игроков.
-

- Движение плавное благодаря интерполяции (сглаживание сетевых задержек).

4. Удалённые вызовы процедур (RPC)

- Реализованы ServerRpc и ClientRpc (NGO) или [ServerRpc]/[ObserversRpc] (FishNet) или аналог Photon.
- Ключевые игровые действия (пример: стрельба, взаимодействие, использование предметов) обрабатываются на сервере.
- Клиент отправляет запрос — сервер валидирует и выполняет — результат рассылается всем клиентам.

5. Игровой цикл и управление сессией

- Реализован полный жизненный цикл сессии: Лобби (ожидание игроков) → Игра → Результаты.
- Сервер управляет переходами между состояниями и отслеживает количество подключённых игроков.
- По завершении матча отображается экран результатов с возможностью перезапуска или выхода.
- Корректная обработка подключения и отключения игроков в процессе игры.

6. Архитектура и структура кода

- Код декомпозирован как минимум по основным блокам: сетевая логика, игровая логика, UI, ввод.
- Не допускается монолитный подход — вся логика не должна быть сосредоточена в одном скрипте.
- Студент понимает назначение своих классов и может объяснить принятые архитектурные решения.
- Приветствуется использование паттернов (Singleton для NetworkManager, Observer для событий и т.п.).

7. Визуальная завершённость и UI

- Продуманное оформление сцены (окружение, освещение, материалы).
- UI отображает сетевую информацию (пример): список подключённых игроков, HP, статус подключения, результаты.
- Никнейм игрока отображается над персонажем.
- Стилй интерфейса цельный и соответствует тематике проекта.

8. Стабильность и производительность

- Минимальная производительность — не ниже 30 FPS (рекомендуется встроить счётчик).
 - Нет рассинхронизации при подключении и отключении игроков.
 - Приложение работает без критических ошибок (вылетов, зависаний).
 - Допускаются не критичные ошибки, не влияющие на сетевой геймплей.
-

9. Звуковое сопровождение

- Звуки действий (например, движение, взаимодействие, столкновение).
- Фоновая музыка (опционально).
- Отсутствие дублирования и артефактов звука.

Дополнительные требования (бонусные)

Выполнение данных пунктов не является обязательным, но даёт преимущество при защите проекта:

- Клиентское предсказание (Client-side Prediction) — мгновенный отклик на действия игрока при наличии задержки.
- Сборка и запуск Dedicated Server под Linux (Headless-режим).
- Использование FishNet/Photon вместо или в дополнение к NGO.
- Звуковое сопровождение: сетевая синхронизация звуков (выстрелы, шаги, взаимодействия) через RPC.
- Использование оборудования кафедры (VR-сфера, кресло с рулём, VR-шлемы) для демонстрации сетевого проекта.

Документация и структура сдачи проекта

При сдаче проекта студент подготавливает отдельную папку, включающую:

- **Краткое описание проекта:** идея, сетевая архитектура (NGO/FishNet/Photon, Host/Dedicated), основные механики, используемые технологии.
- **Список собственных скриптов и их назначение** (с указанием сетевых компонентов: RPC, NetworkVariable и т.д.).
- **Файлы проекта (в архиве):** Assets, Packages, ProjectSettings.
- **Собранный билд (в архиве).**
- **Видео-демонстрация** с записью сетевой игры (минимум 2 окна/клиента одновременно на экране).
- **Итоговое название папки:** *ФИО_Мультиплеер*

Проект должен быть готов к демонстрации сетевой игры на защите: два клиента подключаются и играют в реальном времени (локальная сеть или localhost).

Критерии оценивания

Итоговая аттестация по дисциплине — зачёт. Оценка формируется по 100-балльной шкале на основе следующих критериев:

Критерий	Описание	Макс.	Мин.
1. Сетевая архитектура и подключение	Реализовано подключение (Host/Client или Dedicated Server). NetworkManager настроен корректно. Игроки успешно подключаются и видят друг друга.	20	10
2. Синхронизация данных	NetworkVariable или аналог FishNet используется для синхронизации состояния (HP, никнейм, счёт и т.д.). Данные корректно обновляются на всех клиентах.	15	8
3. Движение и владение объектами	Реализовано сетевое движение персонажей с корректным разделением локального и удалённого игрока. Камера привязана к владельцу. Интерполяция работает.	15	8
4. Удалённые вызовы (RPC)	Реализованы ServerRpc и ClientRpc (или аналоги FishNet). Ключевые действия (стрельба, взаимодействие) обрабатываются на сервере.	15	8
5. Игровой цикл	Реализован полный цикл: лобби/ожидание → игровая сессия → результаты/рестарт. Сервер управляет переходами между состояниями.	10	5
6. Архитектура и качество кода	Код разделён по назначению (сеть, логика, UI, ввод). Нет монолитных скриптов. Студент может объяснить архитектуру проекта.	10	5
7. Визуальная завершенность и UI	Сцена оформлена. UI отображает сетевую информацию (список игроков, HP, статус подключения). Стиль цельный.	5	3
8. Стабильность и производительность	Проект работает без критических ошибок. Нет рассинхронизации при подключении/отключении. Минимум 30 FPS.	5	3
9. Уникальность	Проект содержит собственную механику или идею, выходящую за рамки учебных примеров курса.	5	3
ИТОГО		100	53

Условия получения зачёта:

Зачёт выставляется при одновременном выполнении двух условий:

- Общая сумма баллов ≥ 60 баллов из 100.
- По каждому из 9 критериев набран **минимальный порог** (столбец «Мин.»). Если хотя бы один критерий ниже минимума — зачёт не выставляется, даже при высокой общей сумме.

Парная работа: правила и оценивание

Допускается выполнение проекта в паре из двух человек. Требования к проекту единые для всех форматов работы. Оценка — индивидуальная: на защите преподаватель задаёт вопросы каждому отдельно. Оценка формируется по формуле:

$$\text{Итоговый балл} = \text{Балл за проект (80\%)} + \text{Балл за защиту (20\%)}$$

Балл за проект — общий для пары, рассчитывается по таблице выше и масштабируется до 80 баллов. Балл за защиту — индивидуальный для каждого члена пары, рассчитывается по таблице ниже и масштабируется до 20 баллов. Зачёт выставляется при сумме ≥ 60 баллов и соблюдении минимума по каждому блоку.

Индивидуальные критерии защиты (20 баллов, оценивается отдельно для каждого)

Критерий	Что проверяется на защите	Макс.	Мин.
Знание своей части	Студент может объяснить любой фрагмент своего кода, ответить на вопрос по логике реализации и обосновать принятые решения.	10	5
Знание части партнёра	Студент понимает логику части, написанной партнёром: может объяснить, как она встраивается в общую архитектуру и зачем она нужна.	6	3
Архитектура и решения	Студент может объяснить ключевые архитектурные решения всего проекта в целом: выбор фреймворка, топология, разделение ответственности в паре.	4	2
ИТОГО (защита)	Индивидуальный балл за ответы на защите	20	10

Примечание: если студент не может ответить ни на один вопрос по коду проекта — балл за защиту равен 0 и зачёт не выставляется независимо от балла за проект.